Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251188
Nama Lengkap	ANGELA REVALINE SETIAWAN
Minggu ke / Materi	06 / Struktur Kontrol Perulangan

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Link Github : https://github.com/angelarevalines/71251188_valine.git

1. Definisi Perulangan

Jalannya suatu program dapat diatur secara **sekuensial, percabangan, perulangan**, maupun **gabungan** dari ketiganya. Pengaturan ini biasanya disebut sebagai struktur kontrol. Program membutuhkan suatu perulangan apabila :

1) Melakukan hal sama berulang-ulang.
2) Melakukan sesuatu secara bertahap (step by step) dan setiap tahapnya memiliki langkah yang sama.
3) Mengakses kumpulan data yang berada dalam struktur seperti **list, tuple, queue, stack**, dan lainnya. (List = array / untuk data yang lebih dari 1)

Pada bahasa pemrograman Python, perulangan dapat dilakukan dengan menggunakan **for, while, dan rekursif**. Perulangan rekursif berarti kondisi dimana fungsi akan memanggil dirinya sendiri dan berguna untuk menyelesaikan masalah yang akan dibagi menjadi beberapa inti / fokus masalah.

2. Bentuk Perulangan for

Perulangan for biasanya digunakan dalam kondisi :

- **Jumlah perulangan sudah diketahui sejak awal.**
  Contoh : akan dilakukan pembacaan data dari 10 file teks. Meski setiap file teks memiliki isi yang berbeda, tetapi membaca file teks secara umum tetap sama. Pembacaan akan tetap dilakukan dari file pertama, kedua, dan seterusnya sampai file ke sepuluh.
- **Perulangan terjadi karena operasi yang sama pada suatu rentang data / nilai**.
  Contoh : dalam mencari jumlah dari 100 bilangan pertama, maka secara berturut-turut akan dilakukan penjumlahan 1 + 2 + 3 dan seterusnya sampai bilangan ke 100 yang dilakukan berulang-ulang.

Perulangan for pada rentang tertentu tentu akan lebih mudah dilakukan apabila dengan menggunakan bantuan fungsi **range()**.

- **range(stop)** = untuk menghasilkan rentang dari **0 sampai stop - 1**. Contoh : **range(6)**, berarti menghasilkan rentang 0 - 5.

```
1 for i in range(5, 51):
2     print (i)
```

Gambar 2.1 : Contoh program perulangan yang menggunakan for dan range(stop).

Contoh program tersebut dimulai dari 5 (start) hingga 51 (stop). Dibuat 51 karena program akan berjalan dengan prinsip (stop - 1). Sehingga output akan berhenti di angka 50 tepat.

- **range(start, stop, [step])** = untuk menghasilkan rentang dari **start** sampai **stop** dengan peningkatan sejumlah **step**.

```
for _ in range(5, 51):
    print ("Hai Valine!")
```

Gambar 2.2 : Contoh program perulangan yang menggunakan for dan range(start, stop, [step]) tanpa counter.

Contoh program diatas sama seperti program sebelumnya, hanya saja pada program ini tidak menggunakan counter. Program akan menghasilkan output **Hai Valine!** Sebanyak 50 kali dan tidak membutuhkan nilai counter.

```
for i in range(5, 101, 5):
    print (i)
```

Gambar 2.3 : Contoh program perulangan yang menggunakan for dan range(start, stop, [step]) dengan step positif.

Contoh program diatas akan mengeluarkan bilangan kelipatan 5 dari 5 hingga 100. (5, 10, 15, …, 95, 100)

```
for i in range(100, 0, -5):
    print (i)
```

Gambar 2.4 : Contoh program perulangan yang menggunakan for dan range(start, stop, [step]) dengan step negatif.

Contoh program diatas akan mengeluarkan bilangan kelipatan 5 namun dari angka terbesar hingga terkecil. (100, 95, …, 10, 5)

3. Bentuk Perulangan while

Perulangan while biasanya digunakan pada kondisi dimana jumlah perulangannya belum diketahui sebelumnya. Contoh bentuk perulangan while secara umum :

```
<start>
while <stop>:
    operation
    operation
    <step (optional)>
```

```python
bilangan = 0
genap = False
while genap == False:
    bilangan = int(input('Masukkan bilangan genap: '))
    if bilangan % 2 == 0:
        genap = True
    else:
        print('Bilangan yang kamu masukkan bukan bilangan genap, coba lagi!')
print(bilangan, 'adalah bilangan genap')
```

Gambar 3.1 : Contoh program perulangan while untuk mengecek bilangan genap.

Program diatas akan menentukkan apakah bilangan yang diinput pengguna merupakan bilangan genap. Perulangan yang digunakan adalah **while** karena nilai maksimum dari perulangan ini belum diketahui sebelumnya. Program tersebut juga tidak membutuhkan nilai counter sehingga akan berhenti apabila input yang dimasukkan oleh pengguna adalah bilangan genap.

```
Masukkan bilangan genap: 45
Bilangan yang kamu masukkan bukan bilangan genap, coba lagi!
Masukkan bilangan genap: 69
Bilangan yang kamu masukkan bukan bilangan genap, coba lagi!
Masukkan bilangan genap: 88
88 adalah bilangan genap
```

Gambar 3.2 : Output program while bilangan genap.

```python
i = 8
while i <= 50:
    print(i)
    i = i + 1
```

Gambar 3.3 : Contoh perulangan while.
Program diatas akan menghasilkan output angka 8 hingga 50

4. Penggunaan Break dan Continue

Struktur kontrol perulangan dapat menggunakan **break** dan **continue**.

- **Break** = untuk menghentikan perulangan.
- **Continue** = untuk melanjutkan perulangan ke iterasi berikutnya.

```python
for i in range(1, 11):
    print(i)
print('Done')
```

Gambar 4.1 : Contoh program untuk menampilkan bilangan dari 1 sampai 10.

Pada program diatas, setelah menampilkan bilangan 1 - 10, kemudian pada baris terakhir akan muncul tulisan “Done”.

```
1 for i in range(1, 11):
2     if i == 5:
3         break
4     else:
5         print(i)
6 print('Done')
```

```
1
2
3
4
Done
```

Gambar 4.2 : Contoh program menggunakan **break** dan outputnya.

Pada program diatas, jika nilai i = 5, maka boolean expression dari i == 5 akan menghasilkan nilai True, yang dimana program akan menjalankan instruksi break. Oleh karena itu, perulangan akan dihentikan dan program berlanjut pada baris setelah perulangan, yaitu menampilkan tulisan “Done”.

Continue = diperlukan untuk melewati tahap perulangan sekarang, lalu langsung dilanjutkan ke tahap / iterasi perulangan berikutnya.

```
1 for i in range(1, 11):
2     if i == 5:
3         continue
4     else:
5         print(i)
6 print('Done')
```

Gambar 4.3 : Contoh program menggunakan **continue**.

Program diatas akan menampilkan angka 1 - 10, tapi tanpa menampilkan angka 5 dan pada baris setelah perulangan akan menampilkan tulisan “Done”.

```
1
2
3
4
6
7
8
9
10
Done
```

Gambar 4.4 : Output dari program yang menggunakan continue.

Program akan melewatkan iterasi saat counter (i) bernilai 5.

5. Konversi dari Bentuk for Menjadi Bentuk while

Beberapa hal yang ada pada bentuk **for** dan **while :**

1) Harus ada nilai awal, untuk memulai perulangan.
2) Harus ada nilai akhir, untuk mengakhiri perulangan.
3) Harus ada langkah, agar iterasi dari nilai awal bisa terus berjalan sampai mencapai nilai akhir.

```
1 for i in range(1, 16):
2     print(i)
```

Gambar 5.1 : Contoh perulangan **for**.

Program akan melakukan perulangan dimulai dari 1 hingga berakhir di 15. Namun apabila dikonversi ke bentuk **while**, begini bentuknya :

```
1 i = 1            #awal
2 while i <= 15:   #kondisi akhir
3     print(i)
4     i = i + 1    #step
```

Gambar 5.2 : Perulangan **for** dikonversi ke bentuk **while**.

Sumber Materi : Modul 06 Struktur Kontrol Perulangan (06 - Struktur Kontrol Perulangan)

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

Link Github : https://github.com/angelarevalines/71251188_valine.git

SOAL 1

Source Code :

```
def perkalian(a,b):
    hasil = (a * b)
    proses = ""
    for i in range(1, a + 1):
        proses += str(b)
        if i < a:
            proses += " + "
    print(f"{a} x {b} = {proses} = {hasil}")

perkalian(6, 5)
perkalian(7, 10)
```

Penjelasan :

1. Membuat fungsi bernama **perkalian** yang berisi parameter **a** dan **b**.
2. Hasil perkalian a dan b disimpan dalam variabel **hasil**.
3. Dalam perulangan, variabel proses menyimpan **b** dalam bentuk string.
4. Jika i dalam perulangan kurang dari a maka proses akan menyimpan tanda tambah (+).
5. Menggunakan fungsi print untuk menampilkan proses dan hasil dari variabel **proses** dan **hasil**.
6. Memanggil fungsi **perkalian** untuk menampilkan hasil dengan parameter berupa angka.

Output :

```
6 x 5 = 5 + 5 + 5 + 5 + 5 + 5 = 30
7 x 10 = 10 + 10 + 10 + 10 + 10 + 10 + 10 = 70
```

SOAL 2

Source Code :

```
batas_atas = int(input("Masukkan batas atas = "))
batas_bawah = int(input("Masukkan batas bawah = "))


def ganjil(batas_atas, batas_bawah):
    hasil = []


    if batas_bawah < batas_atas:
        for i in range(batas_bawah, batas_atas + 1):
            if i % 2 == 1:
                hasil.append(str(i))
        print(f"bawah = {batas_bawah}, atas = {batas_atas}.
Karena bawah<atas, berarti dari kecil ke besar, maka hasilnya
adalah:{', '.join(hasil)}.")
    else:
        for i in range(batas_bawah, batas_atas - 1, - 1):
            if i % 2 == 1:
                hasil.append(str(i))
        print(f"bawah = {batas_bawah}, atas = {batas_atas}.
Karena bawah>atas, berarti dari besar ke kecil, maka hasilnya
adalah:{', '.join(hasil)}.")


ganjil(batas_atas, batas_bawah)
```

Penjelasan :

1. Program ini meminta pengguna untuk memasukkan angka **batas_atas** dan **batas_bawah**.
2. Membuat variabel **batas_atas** dan **batas_bawah** untuk menyimpan nilai yang diinput oleh pengguna.
3. Membuat fungsi **ganjil** yang berisi parameter **batas_atas** dan **batas_bawah**.
4. Jika **batas_bawah** < **batas_atas** maka program akan melakukan perulangan **for** dan **range** dengan mengecek bilangan yang berada di dalam **batas_atas** dan **batas_bawah** apakah bilangan ganjil atau bukan (% 2 == 1?). Jika iya, maka bilangan

akan langsung di print, dan lanjut mengecek bilangan berikutnya. Output yang dihasilkan berarti dari kecil ke besar.

5. Selain itu berarti output yang dihasilkan dari besar ke kecil.

Output :

```
Masukkan batas atas = 30
Masukkan batas bawah = 10
bawah = 10, atas = 30. Karena bawah<atas, berarti dari kecil ke besar,
    maka hasilnya adalah:11, 13, 15, 17, 19, 21, 23, 25, 27, 29.
```

SOAL 3

Source Code :

```
def ips(jumlah_matkul):
    sks = 3
    totalnilai = 0
    bobot = 0
    totalsks = sks * jumlah_matkul

    for i in range(jumlah_matkul):
        nilai = str(input(f"Nilai MK {i + 1}: "))

        if nilai == "A":
            bobot = 4
        elif nilai == "B":
            bobot = 3
        elif nilai == "C":
            bobot = 2
        elif nilai == "D":
            bobot = 1
        else:
            print("Input tidak valid! Masukkan berupa
A/B/C/D")
```

```
            return 0

        totalnilai = totalnilai + (bobot * sks)

    if totalsks == 0:
        return 0
    return totalnilai / totalsks

print ("Program penghitung IPS Mahasiswa")
jumlah_matkul = int(input("Berapa jumlah mata kuliah? "))
totalips = ips(jumlah_matkul)
print(f"Nilai IPS anda semester ini {totalips:.2f}")
```

Penjelasan :

1. Membuat fungsi bernama **ips** yang berisi 1 parameter yaitu **jumlah_matkul**.
2. Jumlah sks tiap matkul sama, yaitu 3. Maka kita **set sks = 3**, **totalnilai** dan **totalsks** dimulai dari 0.
3. Total sks dihitung dengan **sks * jumlah matkul**
4. i adalah mata kuliah ke …. i dalam **jumlah_matkul** akan bertambah 1 terus setelah di print sekali.
5. Menggunakan **input** agar pengguna dapat menginput jumlah mata kuliah dan nilai tiap mata kuliah yang disimpan dalam variabel **jumlah_matkul** yang menerima input **integer** dan **nilai** yang menerima input **string**.
6. Seperti tips yang diberikan dalam soal, menggunakan kontrol percabangan dalam perulangan **(if, else)** untuk menentukan bobot pada masing-masing nilai, dari A - D.
7. Program ini bersifat **case sensitif**. Yang mana hanya menerima input nilai A/B/C/D berupa kapital, selain itu program akan mengeluarkan output "Input tidak valid! Masukkan berupa A/B/C/D" dan IPS akan bernilai 0.
8. Menghitung **totalnilai** dengan rumus = **total nilai + (bobot * sks)**
9. Menggunakan pernyataan **if** untuk mengecek apakah **totalsks == 0** atau tidak. Jika iya maka IPS akan bernilai 0.
10. Mengembalikan nilai IPS dengan rumus **totalnilai / totalsks**.
11. Membuat variabel **totalips** untuk menyimpan perhitungan IPS yang dikembalikan oleh fungsi ips dengan parameter **jumlah_matkul**.
12. Output yang diharapkan adalah 2 angka di belakang koma, maka kita menggunakan **:.2f.**

Output :

```
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 6
Nilai MK 1: A
Nilai MK 2: B
Nilai MK 3: C
Nilai MK 4: A
Nilai MK 5: D
Nilai MK 6: C
Nilai IPS anda semester ini 2.67
```

```
Program penghitung IPS Mahasiswa
Berapa jumlah mata kuliah? 2
Nilai MK 1: a
Input tidak valid! Masukkan berupa A/B/C/D
Nilai IPS anda semester ini 0.00
```